

Practical No: - 6

ON

Cryptography and Network Security - SSCS 4020

TITLE: Implementation of the RSA Algorithm (Encryption and Decryption)

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY
(HONOURS.)

SUBMITTED BY:

Name: Shah Rishabh Alpeshbhai

Enrolment: 23SS02IT181

Class: BSCIT-H – 7A - Batch 2023-27

Problem Statement – 1

To implement the RSA algorithm for encrypting and decrypting a given plaintext using public and private keys.

INPUT :

RSA Encryption and Decryption

import math

Input

p = int(input("Enter prime number p: "))

q = int(input("Enter prime number q: "))

message = int(input("Enter message: "))

Calculate n

n = p * q

Calculate Euler's Totient

phi = (p - 1) * (q - 1)

Find e

for e in range(2, phi):

if math.gcd(e, phi) == 1:

break

Find d

for d in range(2, phi):

if (d * e) % phi == 1:

break

Encryption

ciphertext = pow(message, e, n)

```
# Decryption
```

```
decrypted = pow(ciphertext, d, n)
```

```
# Output
```

```
print("\nPublic Key (e, n):", (e, n))
```

```
print("Private Key (d, n):", (d, n))
```

```
print("Ciphertext:", ciphertext)
```

```
print("Decrypted Text:", decrypted)
```

OUTPUT:

```
Enter prime number p: 15
Enter prime number q: 4
Enter message: 151004

Public Key (e, n): (5, 60)
Private Key (d, n): (17, 60)
Ciphertext: 44
Decrypted Text: 44
```

Problem Statement – 2

Aim: To simulate secure communication between a sender and receiver using the RSA public-key cryptosystem.

INPUT :

```
# Secure Communication Using RSA
```

```
import math
```

```
# Receiver generates RSA keys
```

```
p = int(input("Enter prime number p: "))
```

```
q = int(input("Enter prime number q: "))
message = int(input("Enter secret message: "))

# Calculate n and phi
n = p * q
phi = (p - 1) * (q - 1)

# Generate public key e
for e in range(2, phi):
    if math.gcd(e, phi) == 1:
        break

# Generate private key d
for d in range(2, phi):
    if (d * e) % phi == 1:
        break

# Sender encrypts message using receiver's public key
encrypted_message = pow(message, e, n)

# Receiver decrypts using private key
decrypted_message = pow(encrypted_message, d, n)

# Display results
print("\nPublic Key:", (e, n))
print("Private Key:", (d, n))
print("Encrypted Message:", encrypted_message)
print("Decrypted Message:", decrypted_message)
```

OUTPUT:

```
Enter prime number p: 1504
Enter prime number q: 415
Enter secret message: 41005
```

```
Public Key: (5, 624160)
Private Key: (248897, 624160)
Encrypted Message: 94365
Decrypted Message: 563645
```